

=====

=====

=====

Prometheus configuration (kube-prometheus-stack Helm chart)

This documents how Prometheus is configured in this project: how it discovers

and scrapes targets, where the alerting rules live, and the recording rules.

=====

=====

=====

1) SCRAPE CONFIGURATION (how Prometheus finds the app)

With the Prometheus Operator (kube-prometheus-stack), scrape targets are NOT

defined in a static prometheus.yml. Instead they are declared as ServiceMonitor

custom resources, which the Operator turns into scrape configs automatically.

The app's scrape config is defined here:

helm/notes-app/templates/servicemonitor.yaml

Equivalent scrape config it produces:

- job_name: notes-api

selector: app=notes-api # selects the notes-api Service

endpoints:

- port: http # the named Service port

path: /metrics

interval: 15s

The Operator also ships built-in ServiceMonitors that scrape the cluster itself

(node-exporter, kube-state-metrics, kubelet, API server, etc.), which power the

cluster/infrastructure dashboards.

Verify targets are UP: Prometheus UI > Status > Targets (job "notes-api")

2) ALERTING RULES

Defined as a PrometheusRule custom resource (auto-loaded by the Operator):

helm/notes-app/templates/prometheusrule.yaml

Five alerts, each mapped to a failure scenario:

- NotesApiPodCrashLooping (restart rate > 0, for 2m) -> Pod Crash Loop
 - NotesApiHighCPU (cpu > 0.4 cores, for 2m) -> High CPU
 - NotesApiPodNotReady (not ready, for 5m) -> startup/DB issues
 - NotesApiDown (up == 0, for 1m) -> app/DB down
 - NotesApiHighErrorRate (5xx rate > 0.2, for 2m) -> DB connectivity loss
-

View in Prometheus UI > Alerts.

3) RECORDING RULES

Recording rules pre-compute frequently-used or expensive expressions and store

them as new time series, so dashboards and alerts can query the cheaper

pre-aggregated metric instead of recomputing it every time.

These are defined in:

helm/notes-app/templates/prometheusrule.yaml
(recording-rules group)

Example recording rules used in this project:

- record: notes_api:request_rate:sum

expr: sum(rate(flask_http_request_total[5m]))

total request rate across all replicas

- record: notes_api:error_rate:ratio

```
expr: sum(rate(flask_http_request_total{status=~"5.."}  
[5m]))
```

```
/ sum(rate(flask_http_request_total[5m]))
```

```
# fraction of requests that are 5xx (an SLI: error ratio)
```

```
- record: notes_api:cpu_cores:sum_by_pod
```

```
expr:  
sum(rate(container_cpu_usage_seconds_total{namespa  
ce="default", pod=~"notes-api.*"}[3m])) by (pod)
```

```
# per-pod CPU in cores, reused by the High CPU alert  
and the dashboard
```

Benefit: the error-ratio SLI and per-pod CPU are
computed once per evaluation

interval and reused, rather than being recomputed in
every dashboard panel.

4) OTHER PROMETHEUS SETTINGS (set at install time)

```
-----  
-----
```

```
retention: 6h # PoC retention window
```

```
resources.requests.memory: 400Mi
```

resources.limits.memory: 1Gi

service.type: NodePort, nodePort: 30090 # exposed for reviewer access

Installed with:

helm install monitoring prometheus-community/kube-prometheus-stack \

--namespace monitoring \

--set prometheus.prometheusSpec.retention=6h \

--set prometheus.service.type=NodePort \

--set prometheus.service.nodePort=30090 \

... (see docs/ALL-COMMANDS.txt for the full command)
